

Project 8 - Part B

Erin Plaster

Texas A&M Central Texas
MATH-4309-110 Advanced Analysis
Dr. Mienie Scholtz
Decemebr 11, 2025

1 Introduction and Research Question

1.1 Introduction

The goal of this project is to provide junior officers at III Corps with tools that can support situational awareness in the field and tools to assist in predicting logistic needs. Efficient planning requires both the ability to recognize and identify armored vehicles from images and to forecast fuel consumption during convoy missions.

This case study demonstrates how two different datasets can be transformed through preprocessing, explanatory data analysis, visualization, and predictive modeling to prepare officers for missions. The first analysis develops a Random Forest classifier that distinguishes armored from non-armored vehicles using color based image features. The tool demonstrates how machine learning can assist in asset detection through the use of image classification, visualization of the model training, and a short (user appropriate) description of gradient descent. The second analysis builds a multivariate linear regression model to predict fuel consumption based on vehicle weight, speed, and distance traveled. These models illustrate how automated decision systems could work together to support III Corps.

1.2 Research Questions

1. To what extent can a Random Forest image classification model (based on RGB image features) accurately detect armored vs. non-armored vehicles?
2. How accurately can a linear regression model determine the fuel consumption needed for a future military convoy using vehicle weight, average speed, and distance traveled?

2 Data Cleaning and Preprocessing

2.1 Image Data (Classification)

There are two sources for images in the app: a built-in library and a user interface for multiple uploads. A small set of raw images can be located in the following files: `www/armored` and `www/nonarmored`. All of the images follow the same pipeline: images are loaded using the `magick` package, images are resized to 100x100 resolution, then images are resized to standard (R,G,B) arrays. Feature extraction is then applied and the raw images are converted into a set of numerical values (such as mean and standard deviation). This helps reduce the complexity of an image, removes noise, highlights important information, and standardizes input for the classifier. The app created for this project computes the mean and standard deviation of the Red, Green, and Blue channels to create a final vector: $(mean_R, mean_G, mean_B, sd_R, sd_G, sd_B)$ which allows Random Forest (or other ML methods) to operate without deep learning, which can require a heavy time and resource commitment.

2.2 Numeric Data (Linear Regression)

The regression dataset used in this project is a synthetic dataset that consists of 150 simulated convoy trips (to correlate with the project requirements) that each contain vehicle

weight, average speed, distance traveled, and the corresponding fuel consumption. The variables were verified for consistency and realism prior to analysis are intended to mirror a plausible dataset, while maintaining a clean numerical structure for modeling.

3 Exploratory Data Analysis

3.1 Image Feature Exploration

There are six extracted image features (mean and standard deviation of the R,G, and B channels) that were examined to understand how armored and non-armored vehicles differ in basic color characteristics. While armored vehicles have slightly lower mean brightness, and non-armored vehicles tend to have more color variation, there is a lot of overlap which indicates that RGB statistics offer only limited separability. This suggests that the classifier would need to rely mainly on color changes rather than other features to help identify the vehicle.

3.2 Convoy Fuel Dataset Exploration

Exploration of the synthetic numeric dataset indicated a strong positive relationship between fuel consumption and vehicle weight ($r = 0.819$), and a weak-to-moderate relationship between fuel consumption and distance traveled ($r = 0.376$). Average speed had a weak negative relationship between fuel consumption and speed ($r = -0.198$), indicating higher speeds might reduce fuel consumption. All of these numerical indicators support the use of a linear regression model.

4 Statistical/Predictive Models

4.1 Classification Model (Random Forest)

The model uses a Random Forest Classifier for image predictions. What Random Forest does is create an ensemble of decision trees that are trained on a bootstrapped sample of the data [Van16]. The binary splits are then chosen using Gini Impurity to separate the images between armored and non-armored vehicles, and Random Forest predictions are determined by majority vote across the trees. Gradient descent is covered briefly in its own tab and includes a small synthetic dataset to help the user visualize logistic regression using $\hat{y} = \sigma(w_1x_1 + w_2x_2 + b)$, which updates the weights and bias to reduce loss. The gradient descent tab includes a short summary of gradient descent and the mathematical training process for parametric models, which would be important if the user needed to use another type of training model.

4.1.1 Mathematics (Classification)

As stated above, the application computes the mean of R,G,B and standard deviation of R,G,B. The following formulas are used in the classification application:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad \text{mean calculations}$$

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2} \quad \text{standard deviation calculations}$$

$$G = 1 - \sum_{i=1}^k p_i^2 \quad \text{used for Gini Impurity}$$

Both the mean and standard deviation formulas are used to build the data on which Random Forest trains and are crucial to the training model. Additionally, the Random Forest Gini Impurity formula evaluates split equality among the trees. This optimization process is different from Gradient Descent in that at every split in the tree, the model searches for the best possible way to divide the data to reduce impurity. Gini Impurity is combinatorial rather than continuous and with each split it computes which node minimizes impurity in the resulting nodes. This calculation is done repeatedly at every split and the final prediction uses a voting rule, taking on the majority vote.

The gradient descent updates in the model tab are for explanatory purposes and do not update or calculate any inputs in this application. They are included as additional information for the user. The gradient updates are as follows:

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}, \quad b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

Gradient Descent has the following structure:

1. **Initialize:** Choose a beginning point (or random value) and a learning rate, denoted by α .
2. **Compute the gradient:** The first derivative of the function gives the direction and rate of the steepest increase in an optimization problem, or the gradient vector in a multivariable function. Gradient descent will always move towards the minimum point in order to minimize loss or cost, so we take the negative value of the first derivative.
3. **Update the parameter:** We next apply the gradient descent update rule to iteratively adjust x :

$$x_{k+1} = x_k - \alpha f'(x_k)$$

Each step, or iteration, moves the estimate closer to the point that minimizes $f(x)$ (note: if x_k is negative, add rather than subtracting $\alpha f'(x)$, to move x towards the minimum point).

4. **Convergence:** The process repeats until the change is acceptably small, or until $f'(x)$ approaches zero.

4.1.2 Classification Visualizations

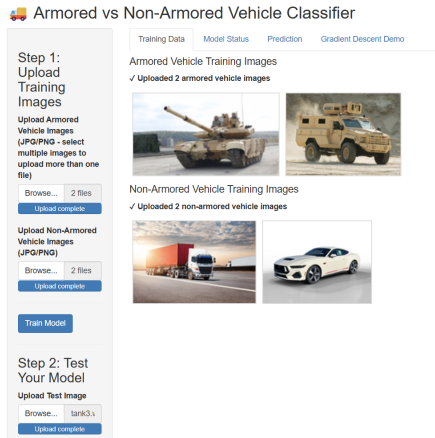


Figure 1: Shiny app user interface.

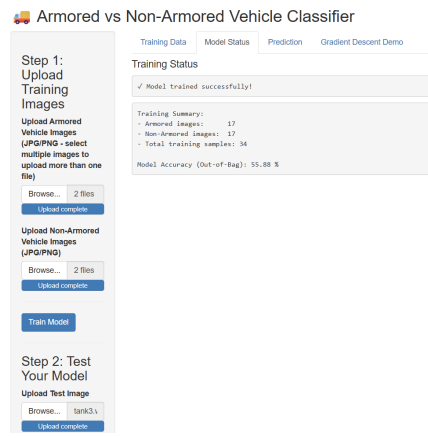


Figure 2: Model Status tab.

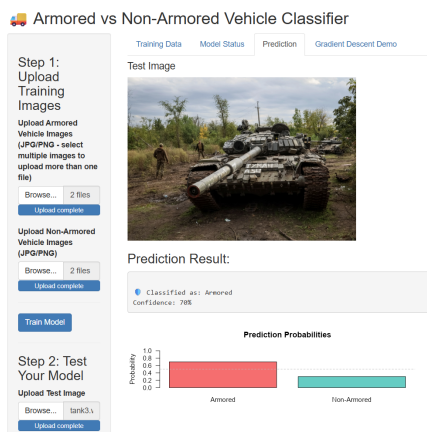


Figure 3: Prediction Tab.

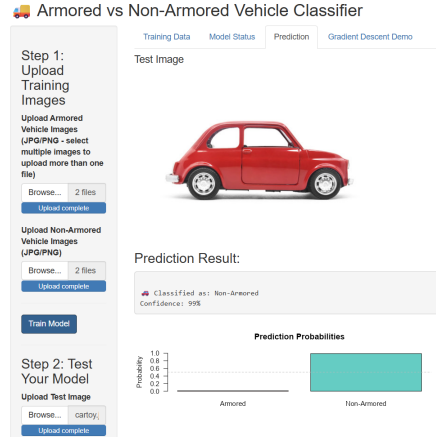


Figure 4: Prediction Tab.

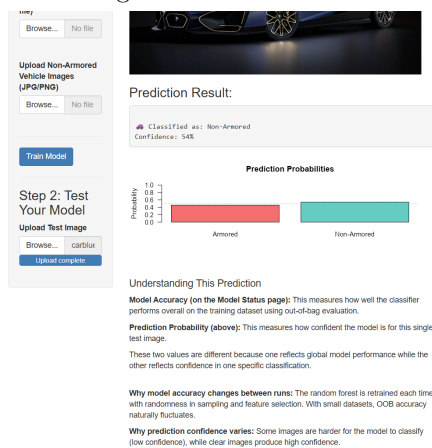


Figure 5: Information below prediction.

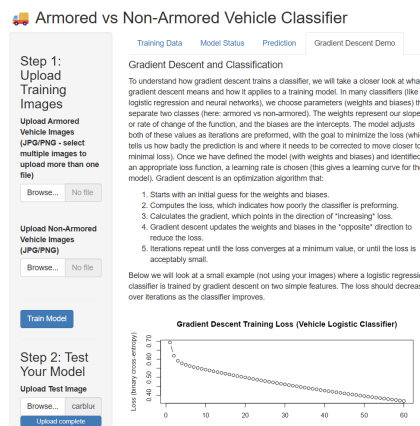


Figure 6: Gradient Descent tab.

4.2 Regression Model (Linear Regression)

Given a dataset containing 150 previous convoy trips:

$$\begin{aligned}x_1 &:= \text{total vehicle weight (tons)} \\x_2 &:= \text{average speed (mph)} \\x_3 &:= \text{distance traveled (miles)} \\y &:= \text{fuel consumed (gallons)}\end{aligned}$$

Build a linear regression model (given):

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

The estimated coefficients from the training data are:

$$\beta_0 = 12.5 \qquad \beta_1 = 3.1 \qquad \beta_2 = -0.8 \qquad \beta_3 = 0.45$$

4.2.1 Coefficient Interpretation (Regression)

Intercept: $\beta_0 = 12.5$

Vehicle Weight Coefficient: $\beta_1 = 3.1$

For every one-unit increase in vehicle weight (one ton), holding average speed and distance constant, the fuel consumption is expected to increase by 3.1 gallons.

Speed Coefficient: $\beta_2 = -0.8$

For every one-unit increase in average speed (one mph), holding vehicle weight and distance constant, the fuel consumption is expected to decrease by -0.8 gallons.

Distance Coefficient: $\beta_3 = 0.45$

For every one-unit increase in distance traveled (one mile), holding vehicle weight and speed constant, the fuel consumption is expected to increase by 0.45 gallons.

Substituting the values into the regression model yields the following:

$$\begin{aligned}\hat{y} &= \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 \\ \hat{y} &= 12.5 + 3.1x_1 - 0.08x_2 + 0.45x_3\end{aligned}$$

The data given for the upcoming convoy is as follows:

$$\begin{aligned}\text{vehicle weight: } &32 \text{ tons} \\ \text{average speed: } &42 \text{ mph} \\ \text{distance traveled: } &85 \text{ miles}\end{aligned}$$

Substituting into the regression model yields the following:

$$\hat{y} = 12.5 + 3.1(32) - 0.08(42) + 0.45(85) = 116.35 \text{ gallons consumed}$$

4.2.2 Implementation in Python

The dataset is loaded into Python and upcoming convoy values can be entered in part 3 (and adjusted as needed). The model displays the prediction, shows basic dataset information, and presents the model equation.

4.2.3 Implementation in R Shiny

The app is a minimal Shiny web application that implements a linear regression prediction workflow. It accepts user inputs for vehicle weight (tons), speed (mph), and distance (miles), then computes fuel consumption using fixed regression coefficients derived from prior convoy data. The application displays the prediction and presents the model equation. This prototype demonstrates how regression models can be used in a simple, browser-based interface for operational planning.

4.2.4 Linear Regression Visualizations

```
weight_tons = 32
speed_mph = 42
distance_miles = 85

# -----
# PART 4: Compute Predicted Fuel Use
# -----
y_pred = (
  b0
  + b1 * weight_tons
  + b2 * speed_mph
  + b3 * distance_miles
)

# -----
# PART 5: Print Results Clearly
# -----
print("Upcoming Convoy Inputs:")
print(f" Vehicle Weight: {weight_tons} tons")
print(f" Average Speed: {speed_mph} mph")
print(f" Distance: {distance_miles} miles")

print("\nPredicted Fuel Consumption:")
print(f" {y_pred:.2f} gallons")

Dataset loaded successfully.
Total convoy trips: 150
Columns: ['weight_tons', 'speed_mph', 'distance_miles', 'fuel_gallons']

Upcoming Convoy Inputs:
Vehicle Weight: 32 tons
Average Speed: 42 mph
Distance: 85 miles

Predicted fuel consumption:
116.35 gallons
```

Figure 7: Python code for the linear regression model.

The screenshot shows a web browser window titled "Convoy Fuel Consumption Predictor". The interface is divided into two main sections. On the left, under "Enter Convoy Inputs:", there are three input fields: "Vehicle Weight (tons)" with the value 32, "Average Speed (mph)" with the value 42, and "Distance Traveled (miles)" with the value 85. Below these is a "Predict Fuel Consumption" button. On the right, under "Predicted Fuel Consumption", there is a text box showing "Predicted Fuel Consumption: 116.35 gallons". Below that, under "Model Equation", there is a text box showing the equation $y = 12.10 + 1.10 \cdot x1 + (-0.00) \cdot x2 + 0.45 \cdot x3$, with definitions for $x1 = \text{weight_tons}$, $x2 = \text{speed_mph}$, and $x3 = \text{distance_miles}$. At the bottom left, a small box indicates "Convoy Dataset Loaded" with "Total Rows: 150" and "Columns: weight_tons, speed_mph, distance_miles, fuel_gallons".

Figure 8: Shiny app user interface for linear regression model.

I used the given initial code scaffolding for the Shiny apps and Python work. I also used OpenAI's ChatGPT to help generate initial code scaffolding for the gradient descent section in the Shiny app [Ope25]. All mathematical explanations, interpretations, and final code modifications are my own work.

5 Interpretation, Accuracy, and Results

5.1 Classification

Out-of-Bag accuracy is used in the app model and provides an internal estimate on the model's performance. Because we have a very small dataset and randomness in training,

accuracy will fluctuate between training sessions. Each time we select to train the model, the results are new and nothing is stored for continued learning. Out-of-Bag accuracy will have moderate accuracy with small datasets, but can be improved if more images are included (particularly if they have different lighting or viewpoints). Variability caused by the random sampling and diversity of the background in the images is present because of the Random Forest classification [IBM21]. We could improve on this by choosing to use a Convolutional Neural Network (CNN) alongside a Random Forest. A CNN would be able to extract lower-level features such as edges, to combine with early layers to create more accurate image predictions [RLM22].

Each test run in the app will include a probability of armored vs. non-armored which reflects a confidence level for that image, not the overall performance of the app. We should expect a relatively high confidence level for very clear images without much background noise and different scores even when the overall model accuracy remains constant. For instance, when testing a tank viewed from the front at ground level, the model may confuse the image with an non-armored vehicle, or drawings with multiple images may return non-armored even though they look like cartoon images of tanks.

5.2 Regression

Weight and distance have the largest positive impact on fuel consumption for the linear regression model, while higher speeds are associated with slightly lower fuel use. The prediction provides a reasonable estimate for planning purposes and can easily be adjusted if needed.

The prediction for the upcoming convoy is 116.35 gallons.

This means that operation management should consider this in the budget and logistics process.

6 Limitations

6.1 Classification

There are limitations to this application. Although the model can generally detect a correct image, the dataset is small and can give varying results between runs. The random forest is also sensitive to outliers and image quality [RLM22]. The user should add as many images as possible to improve stability and generalization, which helps with bootstrapping (sampling datasets with replacement). Another limitation is that the feature set captures only the mean and standard deviation of R,G,B. This means that although color and texture are captured, vehicle shapes, edges and contours, structural features, camouflage patterns, lighting, and shadows are not collected. A larger model would be able to handle these types of attributes. As mentioned above, Random Forest does not learn over time, so that will be another limitation. We will have inaccuracy and a non-linear learning environment because each run is new and random, so we can consider it a discrete learning process. This is expected and how the application is designed to perform. In the experiments, Out-of-Bag accuracy typically ranged between 60% – 80%, with occasional runs outside that range. Because the dataset is small and the Random Forest is retrained each time using randomized sampling, accuracy fluctuates and should be considered an illustrative demonstration rather than a precise detection of real-world accuracy.

6.2 Regression

There are some limitations of the model. It assumes linear variable relationships and does not consider terrain, weather, or vehicle type. The model likely needs more variables in order to be used in a real-life scenario. It is important to note that the predictions would be most accurate within the range of the original dataset as extrapolation can approximate incorrectly and provide incorrect or unrealistic estimates.

7 Conclusion

This case study demonstrated how these models can support junior officers at III Corps by increasing situational awareness in the field and by providing assistance in predicting logistic needs. The classification model helps to distinguish armored from non-armored vehicles, while the linear regression model predicted fuel consumption based on vehicle weight, speed, and distance. Potential improvements to the models include: larger datasets, CNNs(convolutional neural networks) for better feature extraction, multilevel regression models.

References

- [IBM21] IBM. *What is random forest?* Oct. 2021. URL: <https://www.ibm.com/think/topics/random-forest>.
- [Ope25] OpenAI. *ChatGPT*. Sept. 2025. URL: <https://chatgpt.com/>.
- [RLM22] Sebastian Raschka, Yuxi (Hayden) Liu, and Vahid Mirjalili. *Machine Learning with PyTorch and Scikit-Learn : Develop Machine Learning and Deep learning... Models with scikit-learn and pytorch*. S.L.: Packt Publishing Limited, 2022. ISBN: 9781801819312.
- [Van16] Jacob T Vanderplas. *Python data science handbook : essential tools for working with data*. Beijing Etc.: O'reilly, Cop, 2016. ISBN: 9781491912058.